Navigation2 Single Point Navigation

Navigation2 Single Point Navigation

1. Course Content
2. Navigation2 Introduction
3. Running the Cases
   - 3.1 Method 1 — UI Operation
   - 3.2 Method 2 — CLI Command
   - 3.3 Viewing the Costmap
4. Navigation Basic Theory Explanation
   - 4.1 Core Steps of the Navigation Flow
     - 4.1.1 Stage 1: System Initialization and Map Loading
     - 4.1.2 Stage 2: Sensor Data Processing and Environment Perception
     - 4.1.3 Stage 3: Path Planning (Global Planning and Local Planning)
     - 4.1.4 Stage 4: Control Execution and Behavior Decision-Making
   - 4.2 Key Technical Principles
     - 4.2.1 Costmap
     - 4.2.2 AMCL Localization Algorithm
   - 4.3 Path Planning
     - 4.3.1 Global Path Planning
     - 4.3.2 Local Path Planning
5. Navigation Parameter Configuration
   - 5.1 Navigation Speed Debugging
   - 5.2 Obstacle Avoidance Effect Debugging
6. Source Code Analysis

> **Important**
>
> **Note:** This course requires at least one grid map to perform navigation. If you have not built a map, please first refer to the previous SLAM mapping course to build a grid map

1. Course Content

Basic:

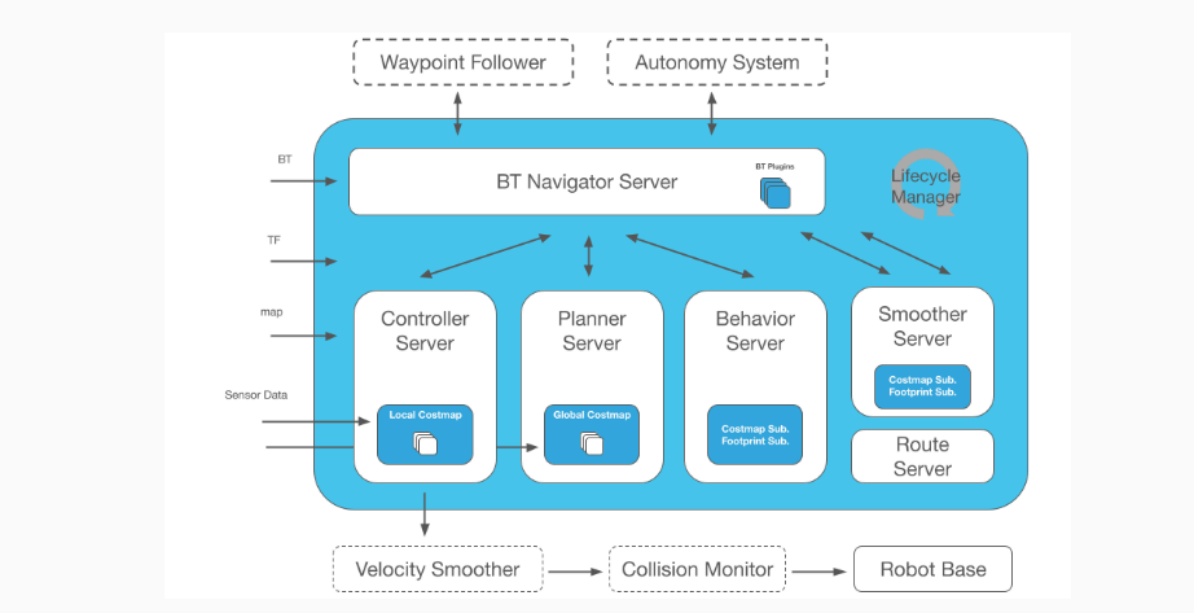
- Run the example program; the map will be loaded in rviz. In the rviz interface, use the 【2D Goal Pose】 tool to give the car a goal point. Combining its own environment, the car will plan a path and move to the destination along the planned path. If it encounters an obstacle along the way, it will automatically avoid it, and stop after reaching the destination.

Advanced:

- Understand the concepts of the global costmap and local costmap, debugging of navigation speed and obstacle avoidance effect, and the principles of the AMCL localization algorithm

2. Navigation2 Introduction

- Navigation2 overall architecture diagram



Navigation2 has the following tools:

- Tools for loading, providing, and storing maps (Map Server)
- Tools for localizing the robot on the map (AMCL)
- Path planning tools for moving from point A to point B while avoiding obstacles (Nav2 Planner)
- Tools for controlling the robot while following a path (Nav2 Controller)
- Tools for converting sensor data into a costmap representation in the robot's world (Nav2 Costmap 2D)
- Tools for building complex robot behaviors using behavior trees (Nav2 Behavior Trees and BT Navigator)
- Tools for computing recovery behaviors when failures occur (Nav2 Recoveries)
- Tools for following sequential waypoints (Nav2 Waypoint Follower)
- Tools and watchdog for managing server lifecycles (Nav2 Lifecycle Manager)
- Plugins that enable user-defined algorithms and behaviors (Nav2 Core)

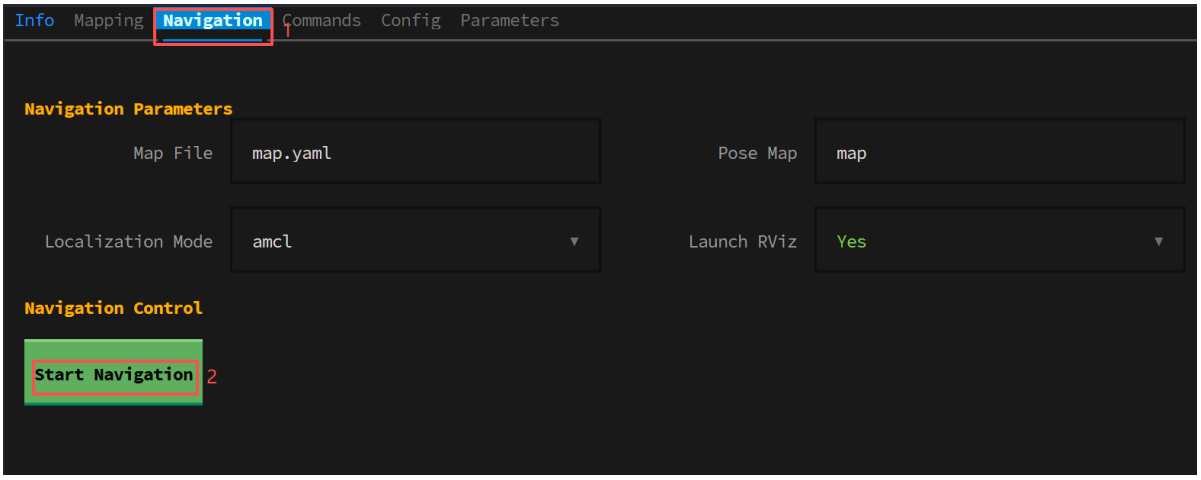
Navigation 2 (Nav 2) is the navigation framework built into ROS 2, whose purpose is to enable a mobile robot to move from point A to point B in a safe manner. Therefore, Nav 2 can perform dynamic path planning, compute motor velocities, avoid obstacles, and execute recovery behaviors.

Nav 2 uses behavior trees (BT) to call modular servers to accomplish an action. Actions can be computing a path, control efforts, recovery, or other navigation-related actions. These actions are all independent nodes that communicate with the behavior tree (BT) through action servers.

3. Running the Cases

3.1 Method 1 — UI Operation

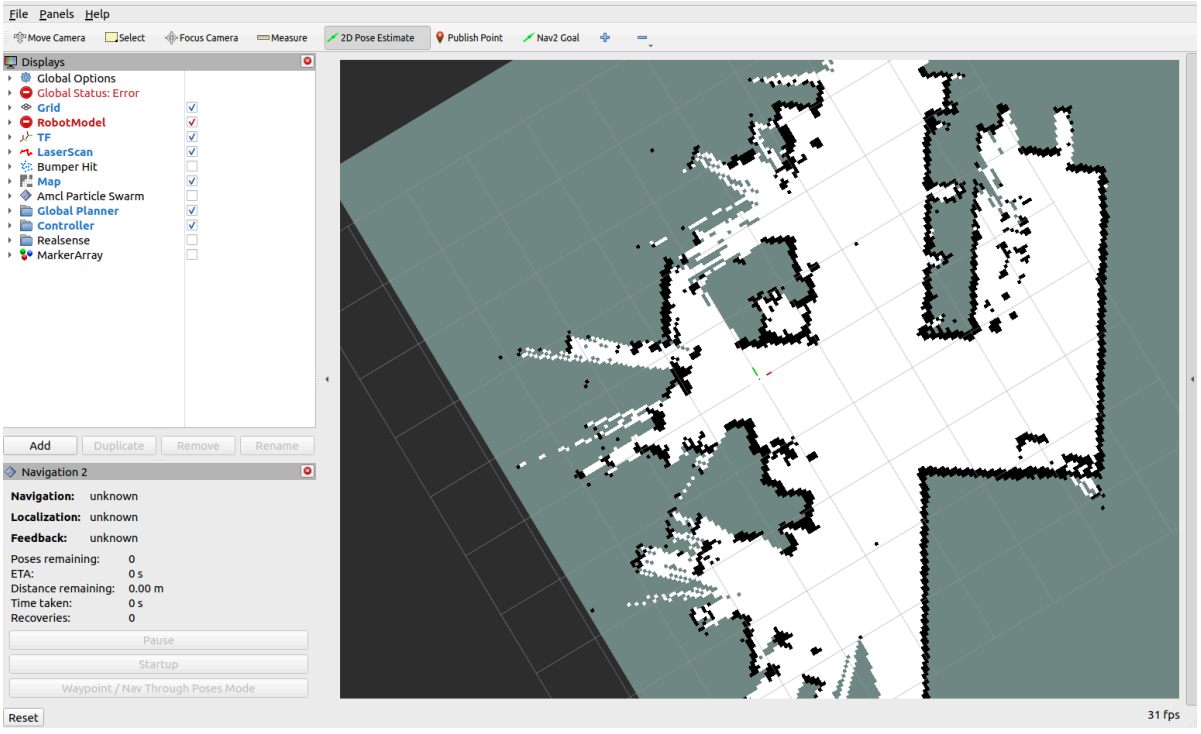
- On RobotManager's **Navigation** page, click the **Start Navigation** button



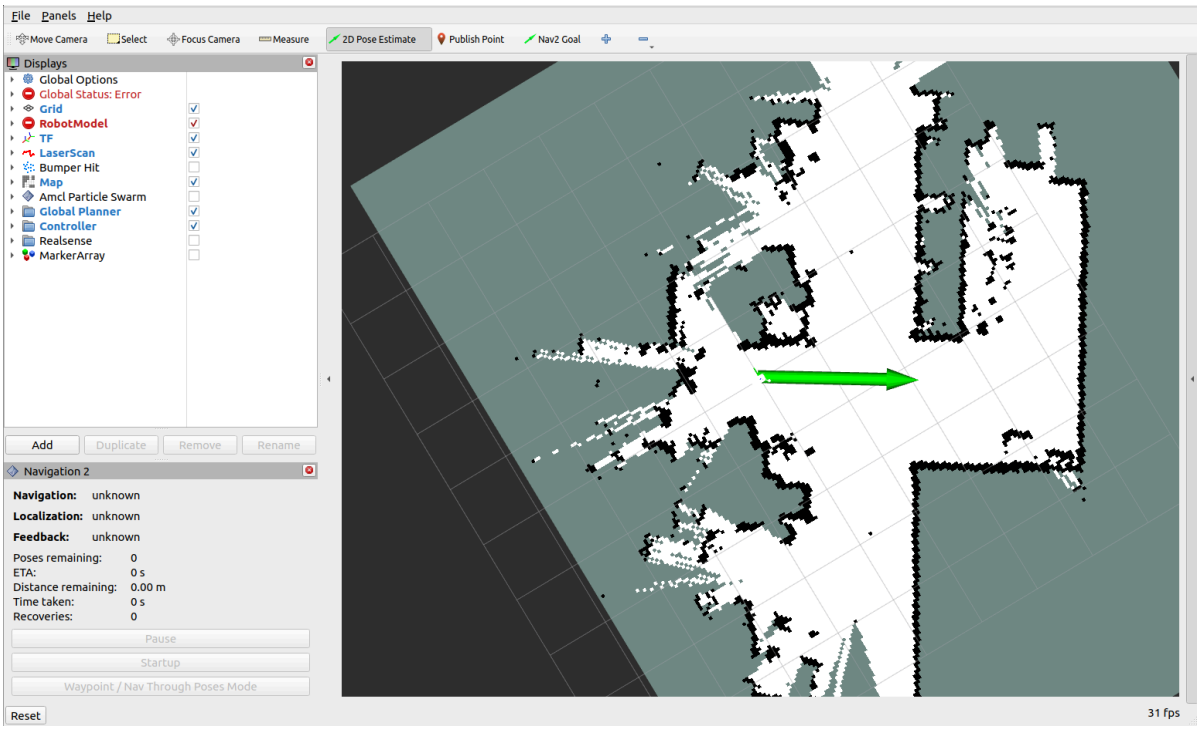
3.2 Method 2 — CLI Command

```
ros2 launch yahboomcar_bringup nav2.launch.py
```

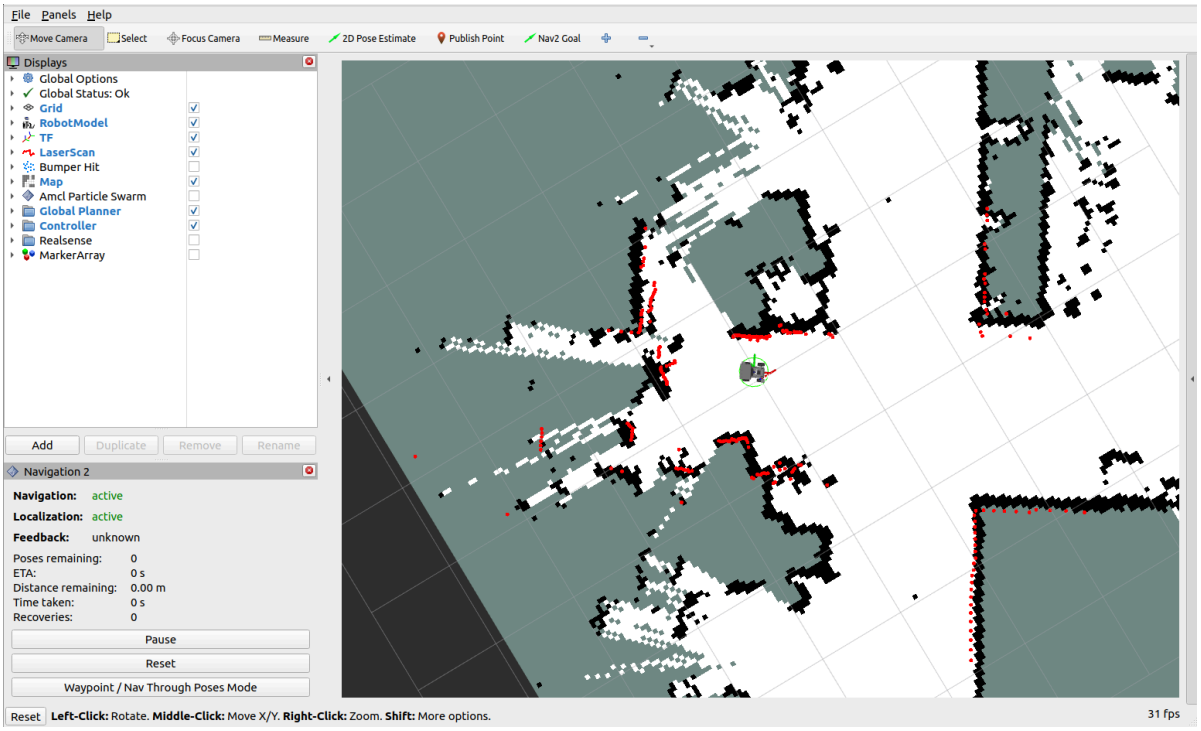
> **Note**
>
> **Optional launch parameter description:**
>
> - map
>   - Map name, default value `map.yaml`; the map must be saved in the system's `~/map` directory.
> - localization_mode
>   - Localization method, default is amcl; options are cartographer localization, slam_toolbox localization, and rtabmap localization



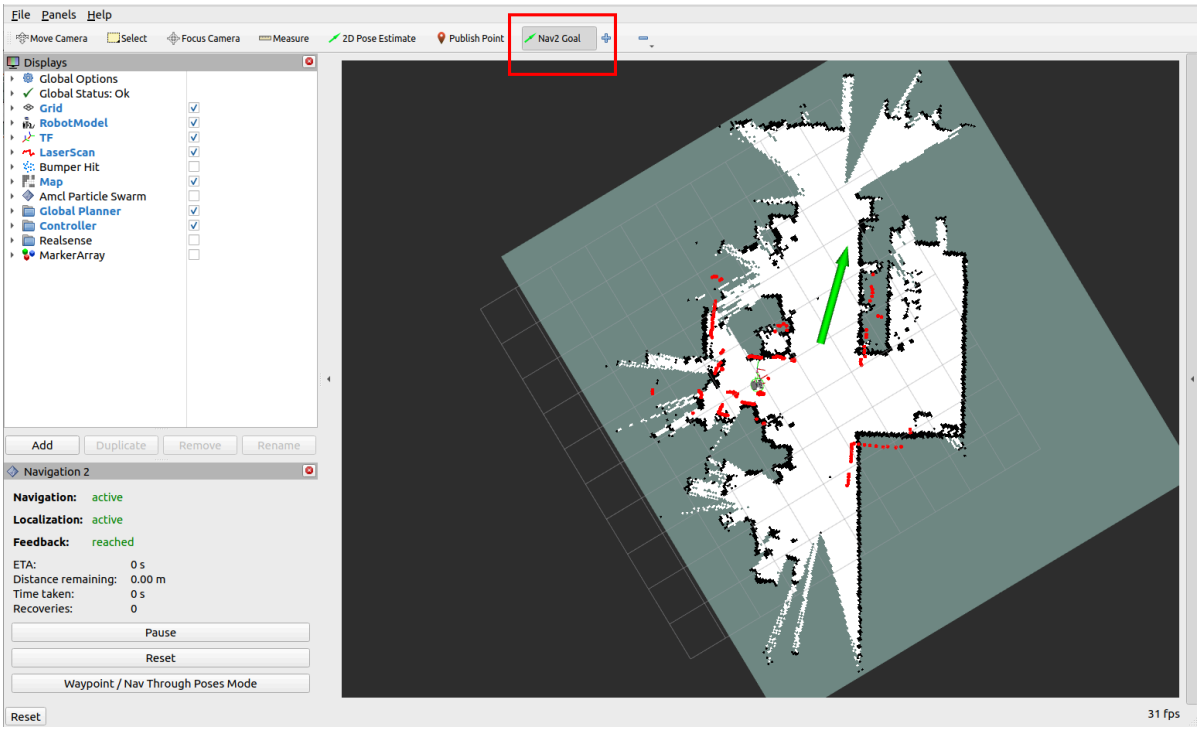
- At this point, you can see that the map has been loaded. Then click 【**2D Pose Estimate**】 above the toolbar to set the car's initial pose. According to the car's position in the actual environment, click and drag with the mouse in rviz, and the car model will move to the position we set. As shown below, if the area scanned by the LiDAR roughly overlaps with the actual obstacle, it means the pose is accurate.



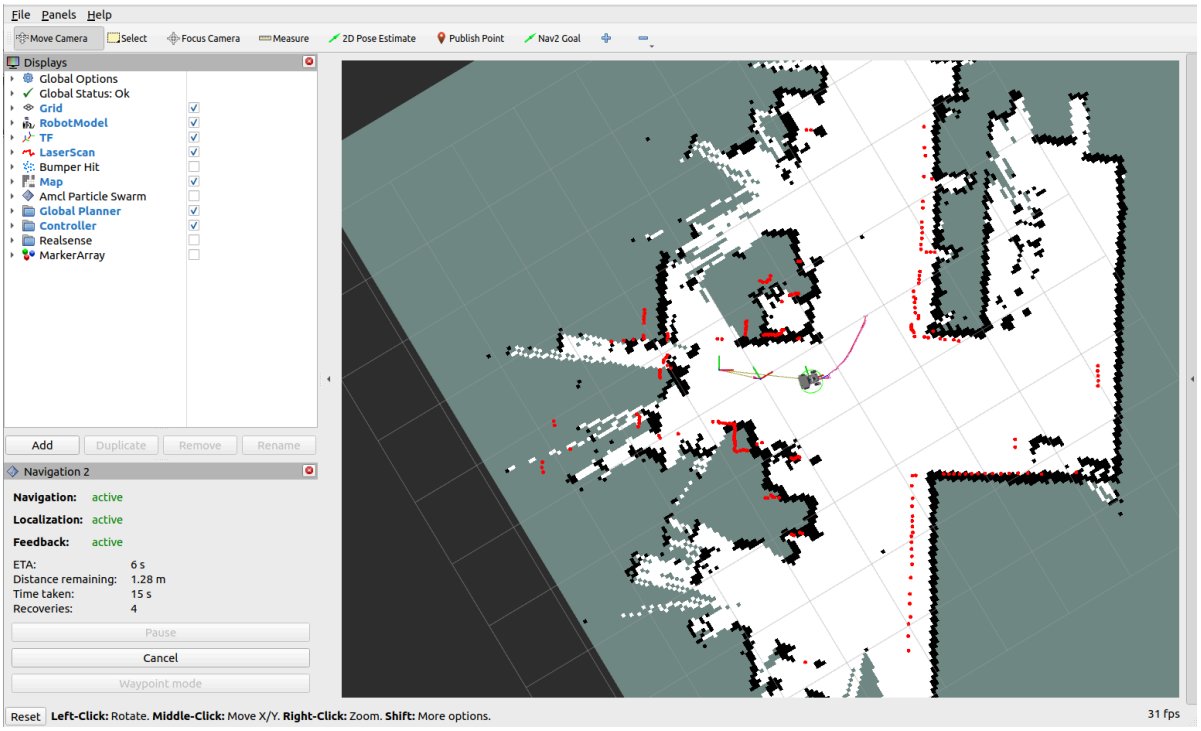
- After pose initialization is complete, the robot model and the red LiDAR 2D point cloud will appear in the rviz interface



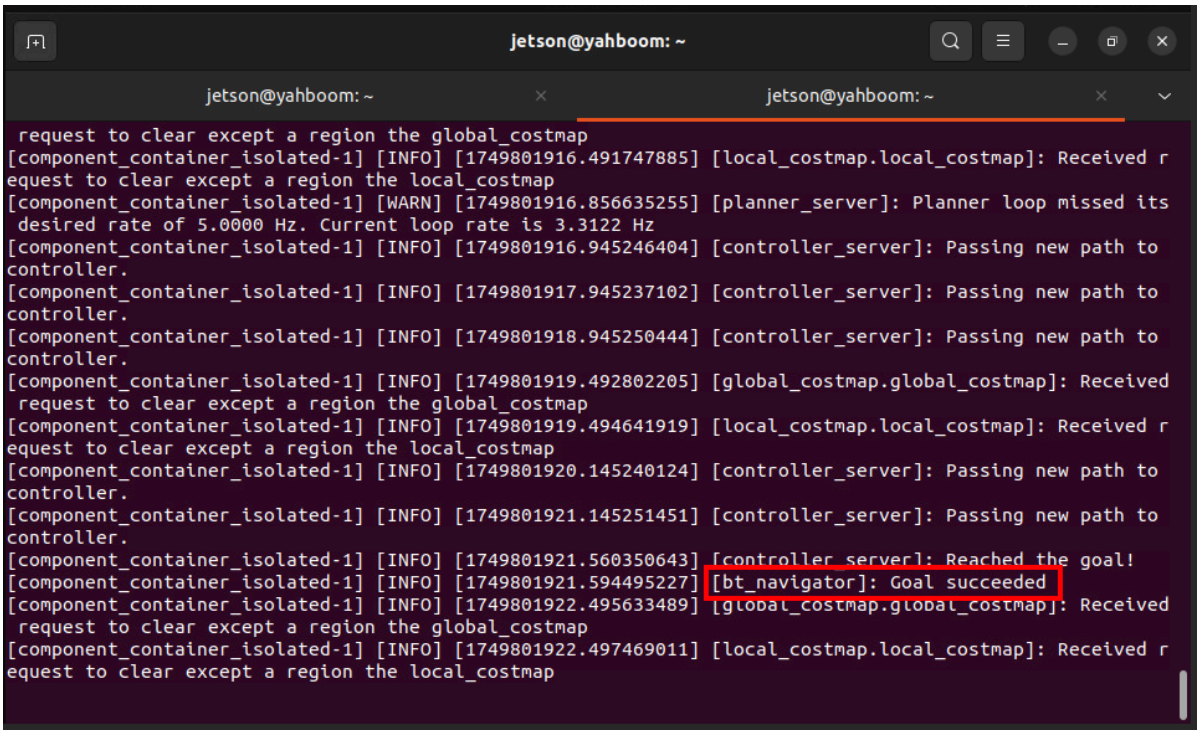
- Single point navigation: click the 【2D Goal Pose】 tool, then use the mouse to select a goal point and goal orientation in rviz, then release



- Combining the surrounding situation, the car plans a path and moves to the goal point along the path.

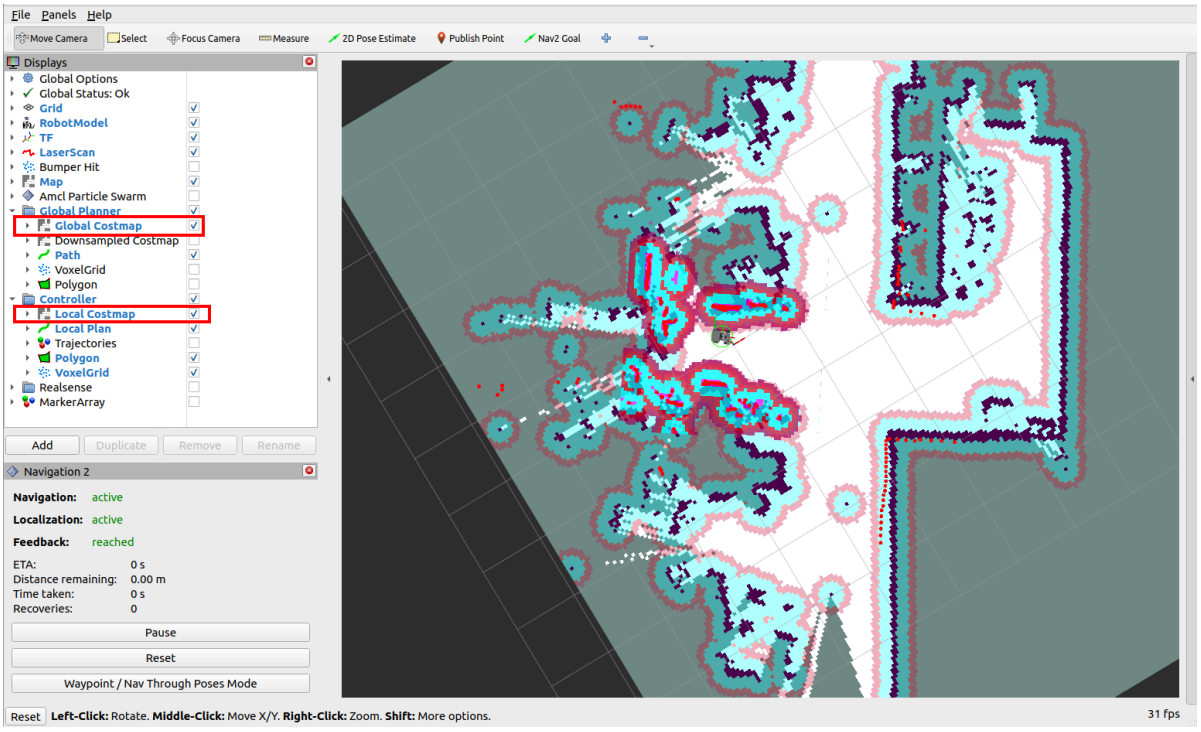


- After the robot successfully reaches the goal point, the car's terminal will display Goal succeeded, indicating successful navigation



3.3 Viewing the Costmap

- If you need to observe the global costmap and local costmap, find Global Costmap under the Global Planner group in the left configuration panel and check it to display the global costmap.
- Similarly, find the Local Costmap option under the Controller group and check it to display the local costmap.



4. Navigation Basic Theory Explanation

4.1 Core Steps of the Navigation Flow

4.1.1 Stage 1: System Initialization and Map Loading

Map Acquisition

- Loads the pre-built grid map from `map_server`
- The map contains static obstacle information and serves as the basic environment model for navigation.

Costmap Initialization

- **Global Costmap**: based on the static map, used for global path planning.
- **Local Costmap**: fuses the LiDAR's `scan` topic in real time for dynamic obstacle avoidance.

4.1.2 Stage 2: Sensor Data Processing and Environment Perception

Sensor Data Input

- LiDAR (`/scan`), IMU (`/imu/data`), and odometry (`/odom`) data are input through ROS topics.

Costmap Update

- The local costmap updates dynamic obstacle information in real time; each grid cell computes an occupancy probability and cost based on sensor data (for example, the closer to an obstacle, the higher the cost).
- Inflation: expands the edges of obstacles to prevent the robot from getting too close to them.

4.1.3 Stage 3: Path Planning (Global Planning and Local Planning)

Global Path Planning

- Inputs: start point (the robot's current pose), end point (the goal pose), and the global costmap.
- The Dijkstra planner algorithm generates the optimal path from the start point to the end point (a series of discrete coordinate points).

Local Path Planning and Tracking

The local planner TEB LocalPlanner generates short-term control commands that the robot can execute, based on the global path and the local costmap.

4.1.4 Stage 4: Control Execution and Behavior Decision-Making

Controller Output

- The controller converts the local path into the robot's linear velocity (`linear.x`) and angular velocity (`angular.z`), and publishes them through the `/cmd_vel` topic.
- Velocity smoothing: avoids abrupt changes in the robot's motion and improves stability.

Behavior Tree Decision Flow

The behavior tree defines the priority and state transition logic of the navigation task. A typical flow is:

1. **Check whether the goal is reachable**: if global planning fails, trigger a "recovery behavior" (such as re-planning).
2. **Perform local obstacle avoidance**: when the local costmap detects an obstacle, temporarily deviate from the global path.
3. **Reach the goal**: when the error between the robot's pose and the goal pose is less than the threshold, the task is complete.

Recovery Behavior Mechanism

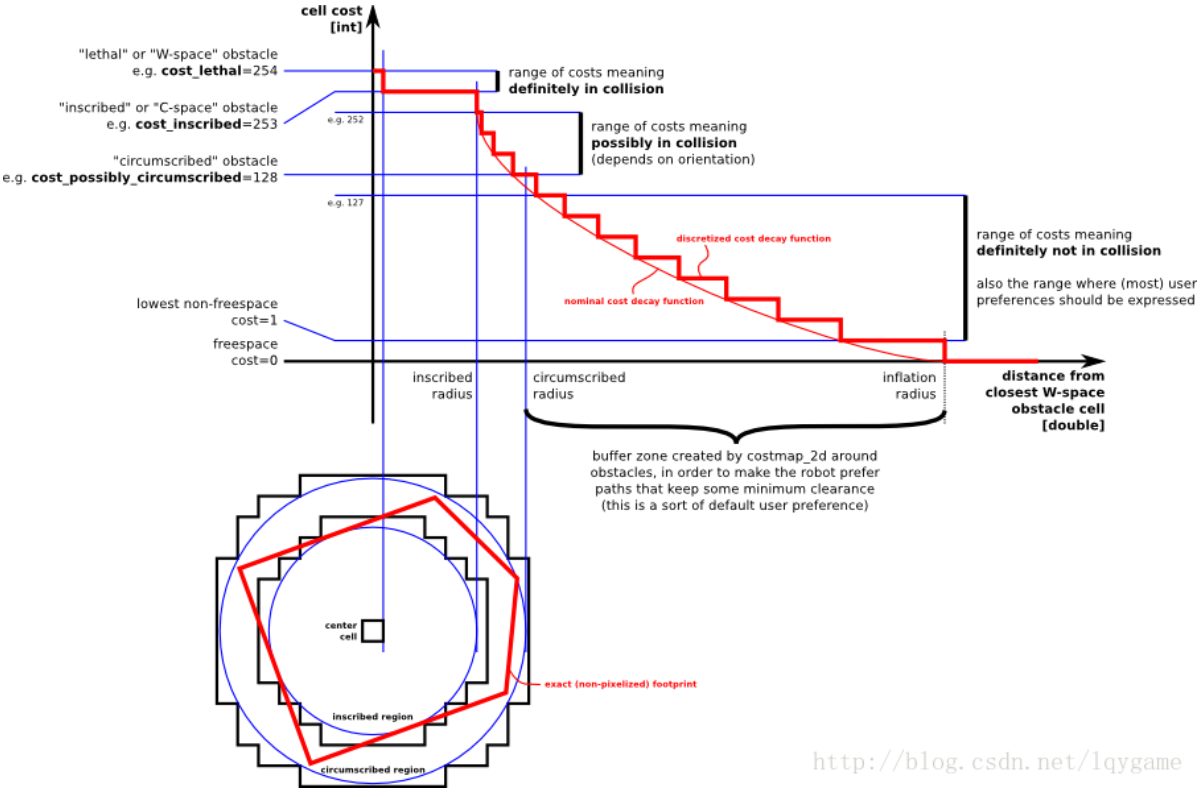
When navigation encounters an exception (such as a completely blocked path), a preset recovery strategy is triggered (such as rotating to search for a new path, or backing up and re-planning).

4.2 Key Technical Principles

4.2.1 Costmap

- **Probabilistic grid representation**: each grid cell stores an occupancy probability (0-1), updated through sensor data (such as LiDAR ray casting).
- **Multi-layer cost superposition**: static cost (obstacles in the map) + dynamic cost (obstacles detected by real-time sensors) + inflation cost (safety distance).

The costmap is a 2D or 3D map that the robot builds and updates by collecting sensor information; you can get a brief understanding from the figure below.

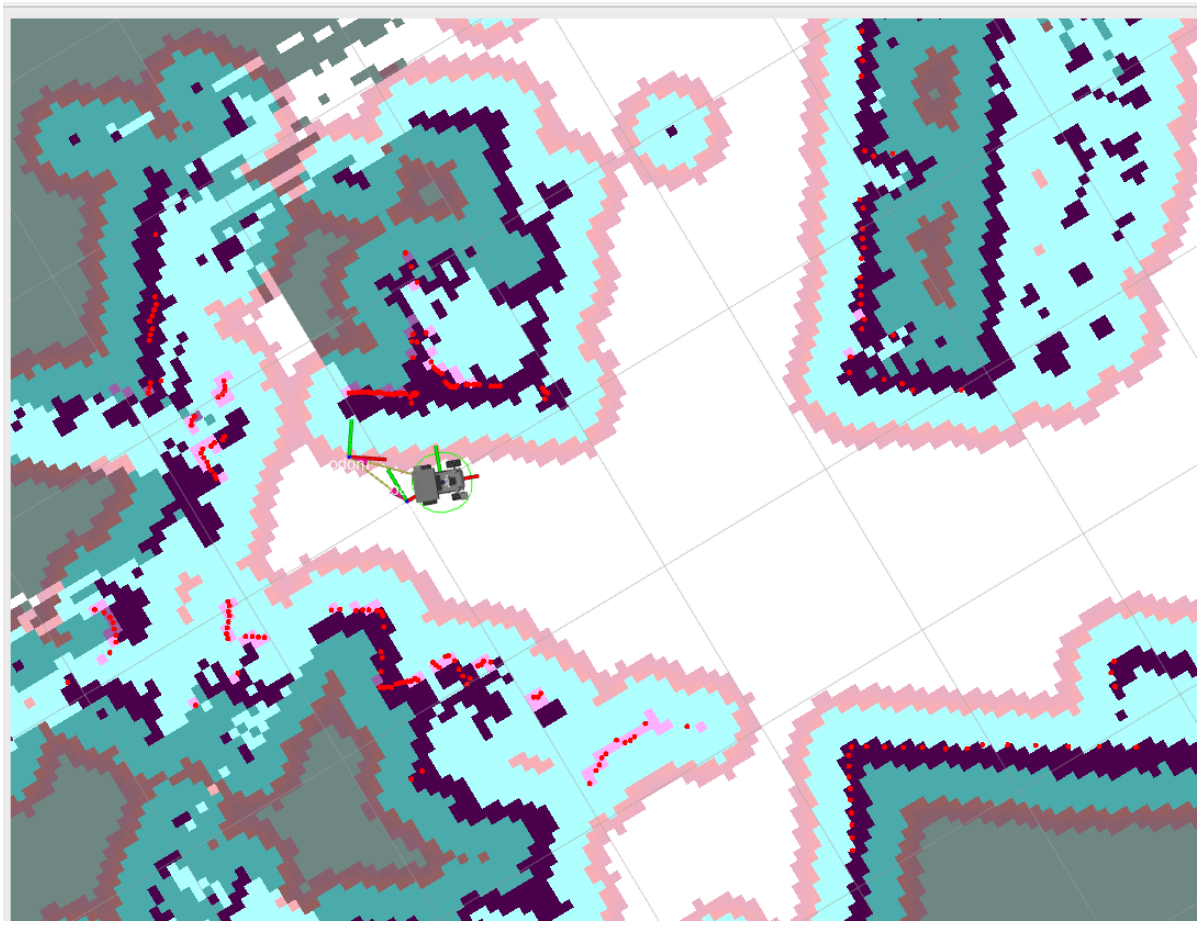


In the figure above, the red parts represent obstacles in the costmap, the blue parts represent the inflated obstacles created by the robot's inscribed circle radius, and the red polygon is the footprint (the vertical projection of the robot's outline). To avoid collisions, the footprint should not intersect the red parts, and the robot's center should not intersect the blue parts.
The ROS costmap uses a grid format, and each cell's value (cell cost) ranges from 0 to 255. It is divided into three states: occupied (has an obstacle), free space (no obstacle), and unknown space; The specific states and their corresponding values are shown in the figure below:

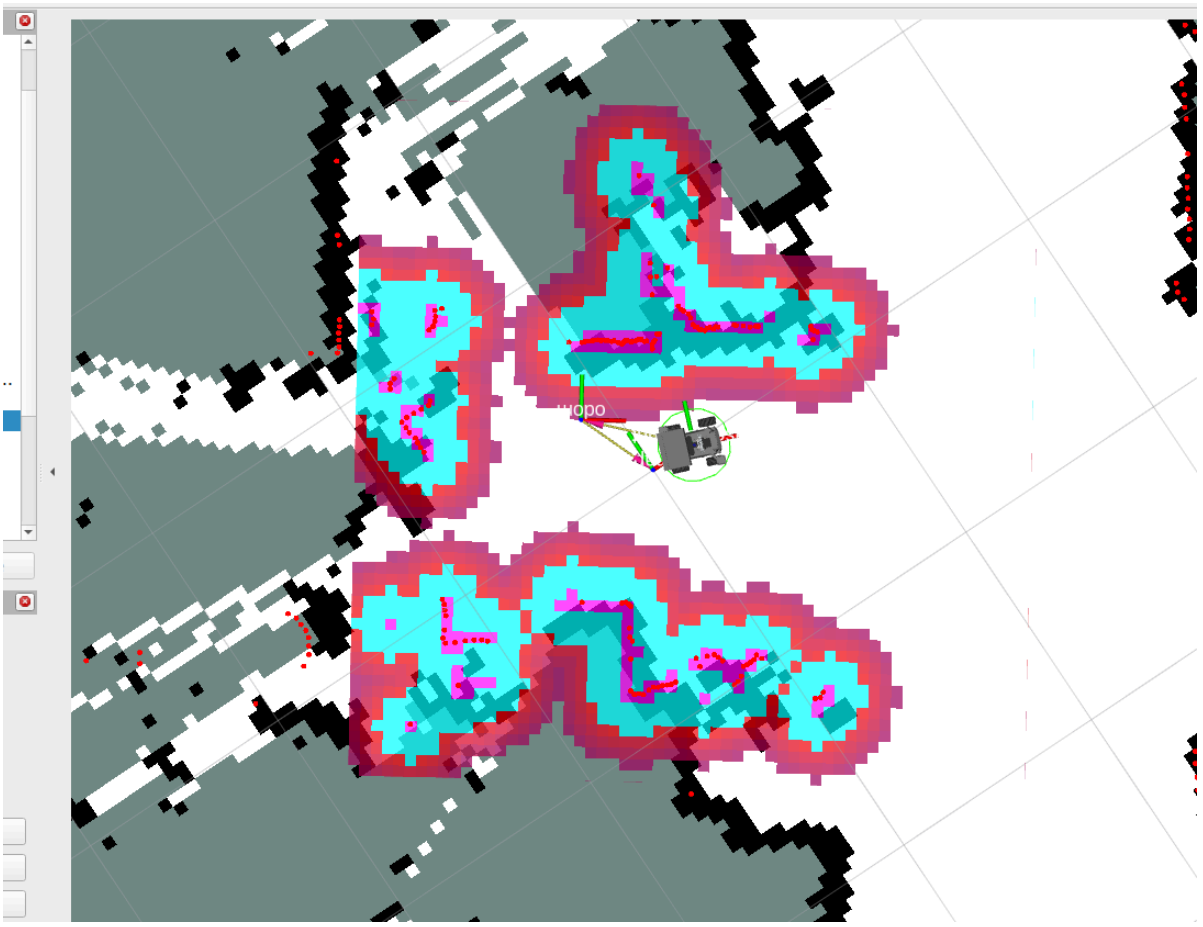
The figure above can be divided into five parts, in which the red polygon area is the robot's outline:

- Lethal: the robot's center coincides with the center of this grid cell, in which case the robot necessarily conflicts with an obstacle.
- Inscribed: the cell's circumscribed circle is inscribed with the robot's outline, in which case the robot also necessarily conflicts with an obstacle.
- Possibly circumscribed: the cell's circumscribed circle is circumscribed with the robot's outline, in which case the robot is essentially leaning near an obstacle, so it does not necessarily conflict.
- Freespace: space with no obstacles.
- Unknown: unknown space.

Global costmap:



Local costmap:

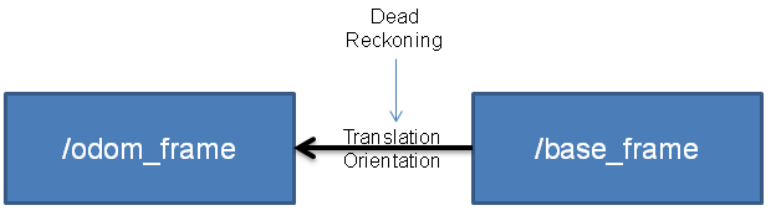


4.2.2 AMCL Localization Algorithm

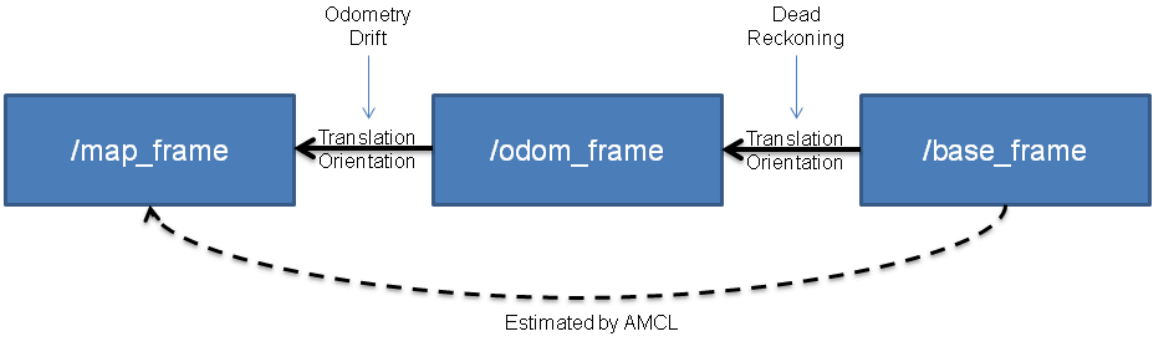
AMCL (Adaptive Monte Carlo Localization) is a probabilistic localization system for 2D mobile robots. It implements the adaptive (or KLD-sampling) Monte Carlo localization method and can use a particle filter to estimate the robot's position based on an existing map.

As shown in the figure below, if the odometry had no error, in a perfect situation we could directly use the odometry information (upper figure) to estimate the position of the robot (base_frame) relative to the odometry coordinate frame. But in reality, the odometry has drift and a non-negligible accumulated error, so AMCL uses the method shown in the lower figure: it first initially localizes base_frame based on the odometry information, then obtains base_frame relative to map_frame (the global map coordinate frame) through the measurement model, thereby knowing the robot's pose in the map. (Note that although what is estimated here is the base-to-map transform, what is ultimately published is the map-to-odom transform, which can be understood as the odometry's drift.)

Odometry Localization



AMCL Map Localization

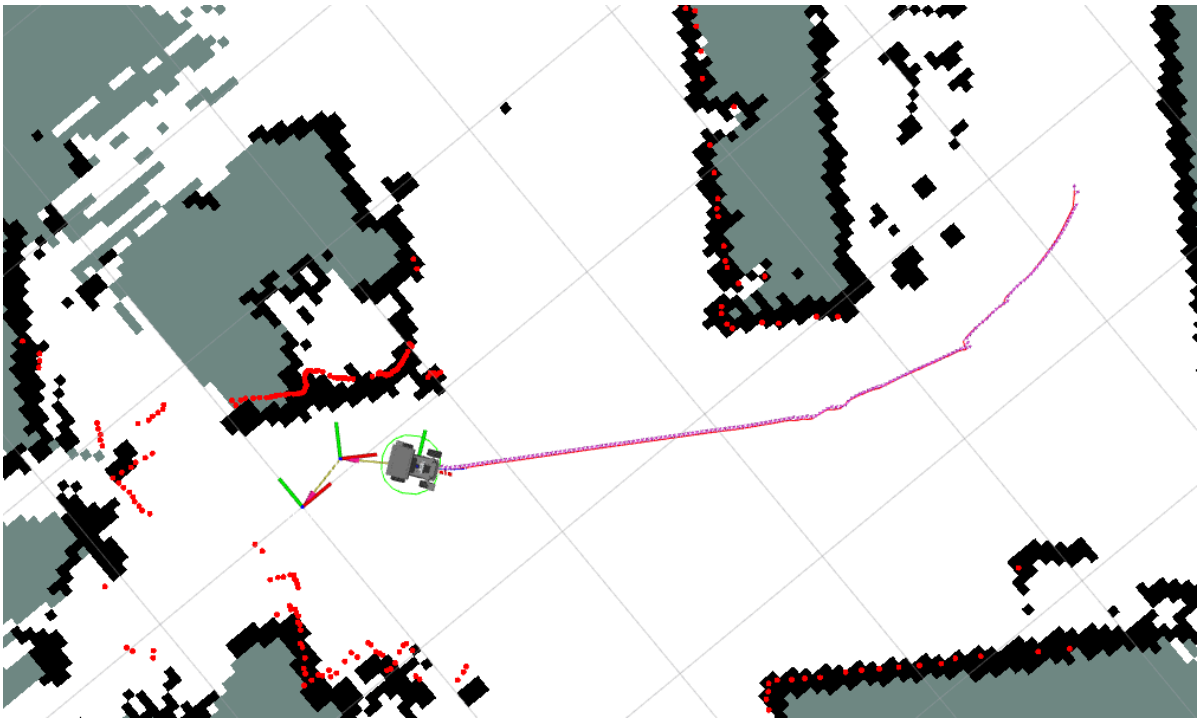


In the TF tree during robot navigation, the coordinate transform between map (map coordinate frame) and odom (odometry coordinate frame) is published by the AMCL localization algorithm, while the coordinate transform between odom (odometry coordinate frame) and base_footprint (the projected coordinate frame of the robot chassis's center point) is published by the **ekf_filter_node** node under the **robot_localization** package, corresponding to the content of the previous **Robot EKF State Estimation** chapter

4.3 Path Planning

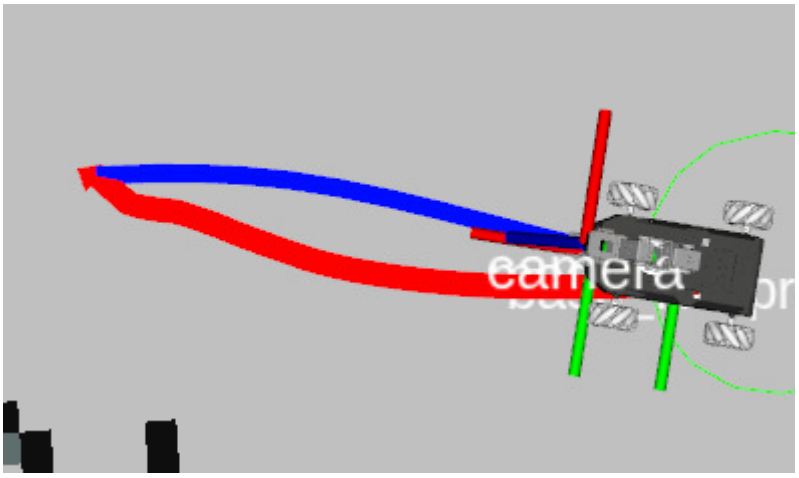
4.3.1 Global Path Planning

- The role of global path planning is to compute an optimal or feasible global path based on the start point and end point in a known environment map, without considering real-time obstacles, aiming for global optimality. It computes the path from the start point to the end point; as shown below, the path formed by connecting the arrows is the global path.



4.3.2 Local Path Planning

- The role of local path planning is to dynamically adjust the path based on real-time LiDAR sensor data during the robot's motion, avoiding obstacles (obstacles will be displayed in the costmap). As shown below, the blue path is the locally planned path and the red is the global path. (The factory-default MPPI controller does not display the path; only the TEB or DWB controller displays the local path.)



5. Navigation Parameter Configuration

- Configuration file path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/param/nav2_params.yaml
```

5.1 Navigation Speed Debugging

Parameter Adjustment Guide

- If you need to adjust the movement speed and rotational angular velocity during navigation, adjust `max_vel_x` and `max_vel_theta`
- If you need to adjust the precision of reaching the goal point, adjust `xy_goal_tolerance` and `yaw_goal_tolerance`
- See the comments below for other parameter adjustments

5.2 Obstacle Avoidance Effect Debugging

Parameter Adjustment Guide

- If the robot gets too close to obstacles in the map during navigation, you can keep the robot away from obstacles by adjusting the inflation area: increase `inflation_radius` (inflation radius) and `cost_scaling_factor` (scaling factor; the larger the value, the harder it is for the car to approach obstacles) in `local_costmap` and `global_costmap`
- See the comments below for other parameter adjustments

```
# AMCL (Adaptive Monte Carlo Localization) configuration
amcl:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False
    # Noise parameter of the odometry model controlling the robot's rotation
    alpha1: 0.2
    # Noise parameter of the odometry model controlling the translation caused by
the robot's rotation
    alpha2: 0.2
    # Noise parameter of the odometry model controlling the robot's translation
    alpha3: 0.2
    # Noise parameter of the odometry model controlling the rotation caused by
the robot's translation
    alpha4: 0.2
    # Noise parameter of the odometry model controlling the robot's rotation
(another aspect)
    alpha5: 0.2
    # Name of the robot's base coordinate frame
    base_frame_id: "base_footprint"
    # Distance threshold for beam skipping, used in laser scan data processing
    beam_skip_distance: 0.5
    # Error threshold for beam skipping, used to decide whether to skip certain
beams
    beam_skip_error_threshold: 0.9
    # Threshold for beam skipping, used to decide whether to skip certain beams
    beam_skip_threshold: 0.3
    # Whether to enable the beam skipping feature
    do_beamskip: false
    # Name of the global coordinate frame
    global_frame_id: "map"
    # Decay coefficient for short laser reflections
    lambda_short: 0.1
```

```yaml
    # Maximum likelihood distance of the laser scan
    laser_likelihood_max_dist: 2.0
    # Maximum range of the laser scan
    laser_max_range: 100.0
    # Minimum range of the laser scan
    laser_min_range: -1.0
    # Laser model type
    laser_model_type: "likelihood_field"
    # Maximum number of laser beams used per update
    max_beams: 60
    # Maximum number of particles allowed in the particle filter
    max_particles: 2000
    # Minimum number of particles allowed in the particle filter
    min_particles: 500
    # Name of the odometry coordinate frame
    odom_frame_id: "odom"
    # Error threshold of the particle filter
    pf_err: 0.05
    # Confidence threshold of the particle filter
    pf_z: 0.99
    # Fast recovery factor, used in the particle filter's recovery mechanism
    recovery_alpha_fast: 0.0
    # Slow recovery factor, used in the particle filter's recovery mechanism
    recovery_alpha_slow: 0.0
    # Resampling interval
    resample_interval: 1
    # Type of the robot's motion model
    robot_model_type: "nav2_amcl::DifferentialMotionModel"
    # Rate at which the robot's pose is saved
    save_pose_rate: 0.5
    # Standard deviation of a laser hit
    sigma_hit: 0.2
    # Whether to broadcast the TF transform
    tf_broadcast: true
    # Publishing rate of the TF transform
    tf_publish_rate: 10.0
    # Tolerance time of the transform
    transform_tolerance: 0.2
    # Minimum threshold for angular update
    update_min_a: 0.2
    # Minimum threshold for distance update
    update_min_d: 0.25
    # Weight of a laser hit
    z_hit: 0.5
    # Weight of the maximum laser range
    z_max: 0.05
    # Weight of random laser measurements
    z_rand: 0.5
    # Weight of short laser reflections
    z_short: 0.05
    # Topic name of the laser scan data
    scan_topic: scan

# AMCL map client configuration
amcl_map_client:
  ros__parameters:
```

```yaml
    # Whether to use simulation time; False means using actual time
    use_sim_time: False

# AMCL RCLCPP node configuration
amcl_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False

# BT navigator configuration
bt_navigator:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False
    # Name of the global coordinate frame
    global_frame: map
    # Name of the robot's base coordinate frame
    robot_base_frame: base_link
    # Topic name of the odometry data
    odom_topic: /odom
    # Default behavior tree XML file name
    default_bt_xml_filename: "navigate_w_replanning_and_recovery.xml"
    # Behavior tree loop period
    bt_loop_duration: 10
    # Default server timeout
    default_server_timeout: 20
    # Whether to enable Groot monitoring
    enable_groot_monitoring: True
    # Groot's ZMQ publisher port
    groot_zmq_publisher_port: 1666
    # Groot's ZMQ server port
    groot_zmq_server_port: 1667
    # List of behavior tree plugin library names
    plugin_lib_names:
    - nav2_compute_path_to_pose_action_bt_node
    - nav2_compute_path_through_poses_action_bt_node
    - nav2_follow_path_action_bt_node
    - nav2_back_up_action_bt_node
    - nav2_spin_action_bt_node
    - nav2_wait_action_bt_node
    - nav2_clear_costmap_service_bt_node
    - nav2_is_stuck_condition_bt_node
    - nav2_goal_reached_condition_bt_node
    - nav2_goal_updated_condition_bt_node
    - nav2_initial_pose_received_condition_bt_node
    - nav2_reinitialize_global_localization_service_bt_node
    - nav2_rate_controller_bt_node
    - nav2_distance_controller_bt_node
    - nav2_speed_controller_bt_node
    - nav2_truncate_path_action_bt_node
    - nav2_goal_updater_node_bt_node
    - nav2_recovery_node_bt_node
    - nav2_pipeline_sequence_bt_node
    - nav2_round_robin_node_bt_node
    - nav2_transform_available_condition_bt_node
    - nav2_time_expired_condition_bt_node
```

```yaml
    - nav2_distance_traveled_condition_bt_node
    - nav2_single_trigger_bt_node
    - nav2_is_battery_low_condition_bt_node
    - nav2_navigate_through_poses_action_bt_node
    - nav2_navigate_to_pose_action_bt_node
    - nav2_remove_passed_goals_action_bt_node
    - nav2_planner_selector_bt_node
    - nav2_controller_selector_bt_node
    - nav2_goal_checker_selector_bt_node

# BT navigator RCLCPP node configuration
bt_navigator_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False

# Controller server configuration
controller_server:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False
    # Controller operating frequency
    controller_frequency: 5.0
    # Minimum velocity threshold in the X direction
    min_x_velocity_threshold: 0.001
    # Minimum velocity threshold in the Y direction
    min_y_velocity_threshold: 0.5
    # Minimum velocity threshold in the rotational direction
    min_theta_velocity_threshold: 0.001
    # Maximum allowed failure tolerance
    failure_tolerance: 0.3
    # Progress checker plugin name
    progress_checker_plugin: "progress_checker"
    # List of goal checker plugin names
    goal_checker_plugins: ["general_goal_checker"]
    # List of controller plugin names
    controller_plugins: ["FollowPath"]

    # Progress checker parameters
    progress_checker:
      # Progress checker plugin type
      plugin: "nav2_controller::SimpleProgressChecker"
      # Minimum radius the robot must move
      required_movement_radius: 0.5
      # Maximum allowed movement time
      movement_time_allowance: 10.0

    general_goal_checker:
      # Whether it is a stateful goal checker
      stateful: True
      # Goal checker plugin type
      plugin: "nav2_controller::SimpleGoalChecker"
      # Goal tolerance in the X and Y directions
      xy_goal_tolerance: 0.15
      # Goal tolerance in the rotational direction
      yaw_goal_tolerance: 0.15
```

```yaml
    # TEB parameters
    FollowPath:
      plugin: "teb_local_planner::TebLocalPlannerROS"
      footprint_model.type: circular
      footprint_model.radius: 0.1
      min_obstacle_dist: 0.15
      inflation_dist: 0.1
      costmap_converter_plugin: "costmap_converter::CostmapToPolygonsDBSMCCH"
      costmap_converter_spin_thread: True
      costmap_converter_rate: 15
      enable_homotopy_class_planning: True
      enable_multithreading: True
      optimization_verbose: False
      teb_autoresize: True
      min_samples: 3
      max_samples: 20
      max_global_plan_lookahead_dist: 1.0
      visualize_hc_graph: True
      max_vel_x: 0.28
      max_vel_theta: 3.0
      acc_lim_x: 2.5
      acc_lim_theta: 0.25

# Controller server RCLCPP node configuration
controller_server_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False

# Local costmap configuration
local_costmap:
  local_costmap:
    ros__parameters:
      # Costmap update frequency
      update_frequency: 5.0
      # Costmap publishing frequency
      publish_frequency: 2.0
      # Name of the global coordinate frame
      global_frame: odom
      # Name of the robot's base coordinate frame
      robot_base_frame: base_link
      # Whether to use simulation time; False means using actual time
      use_sim_time: False
      # Whether to use a rolling window
      rolling_window: true
      # Costmap width
      width: 3
      # Costmap height
      height: 3
      # Costmap resolution
      resolution: 0.05
      # Robot radius
      robot_radius: 0.15
      # Costmap plugin list
      plugins: ["obstacle_layer", "voxel_layer", "inflation_layer"]
```

```
      inflation_layer:
        # Inflation layer plugin type
        plugin: "nav2_costmap_2d::InflationLayer"
        # Inflation radius
        inflation_radius: 0.3
        # Cost scaling factor
        cost_scaling_factor: 3.0
      obstacle_layer:
        # Obstacle layer plugin type
        plugin: "nav2_costmap_2d::ObstacleLayer"
        # Whether to enable the obstacle layer
        enabled: True
        # Observation source name
        observation_sources: scan
        scan:
          # Topic name of the laser scan data
          topic: /scan
          # Maximum obstacle height
          max_obstacle_height: 2.0
          # Whether to clear obstacles
          clearing: True
          # Whether to mark obstacles
          marking: True
          # Data type
          data_type: "LaserScan"
      voxel_layer:
        # Voxel layer plugin type
        plugin: "nav2_costmap_2d::VoxelLayer"
        # Whether to enable the voxel layer
        enabled: True
        # Whether to publish the voxel map
        publish_voxel_map: True
        # Origin in the Z direction
        origin_z: 0.0
        # Resolution in the Z direction
        z_resolution: 0.05
        # Number of voxels in the Z direction
        z_voxels: 16
        # Maximum obstacle height
        max_obstacle_height: 2.0
        # Marking threshold
        mark_threshold: 0
        # Observation source name
        observation_sources: scan
        scan:
          # Topic name of the laser scan data
          topic: /scan
          # Maximum obstacle height
          max_obstacle_height: 2.0
          # Whether to clear obstacles
          clearing: True
          # Whether to mark obstacles
          marking: True
          # Data type
          data_type: "LaserScan"
          # Maximum range of ray tracing
```

```yaml
          raytrace_max_range: 3.0
          # Minimum range of ray tracing
          raytrace_min_range: 0.0
          # Maximum detection range for obstacles
          obstacle_max_range: 2.5
          # Minimum detection range for obstacles
          obstacle_min_range: 0.0
      static_layer:
        # Whether to subscribe to transient local maps
        map_subscribe_transient_local: True
      # Whether to always send the full costmap
      always_send_full_costmap: True
  local_costmap_client:
    ros__parameters:
      # Whether to use simulation time; False means using actual time
      use_sim_time: False
  local_costmap_rclcpp_node:
    ros__parameters:
      # Whether to use simulation time; False means using actual time
      use_sim_time: False

# Global costmap configuration
global_costmap:
  global_costmap:
    ros__parameters:
      # Costmap update frequency
      update_frequency: 1.0
      # Costmap publishing frequency
      publish_frequency: 1.0
      # Name of the global coordinate frame
      global_frame: map
      # Name of the robot's base coordinate frame
      robot_base_frame: base_link
      # Whether to use simulation time; True means using simulation time
      use_sim_time: True
      # Robot radius
      robot_radius: 0.2
      # Costmap resolution
      resolution: 0.05
      # Whether to track unknown space
      track_unknown_space: false
      # Costmap plugin list
      plugins: ["static_layer", "obstacle_layer", "voxel_layer",
"inflation_layer"]
      obstacle_layer:
        # Obstacle layer plugin type
        plugin: "nav2_costmap_2d::ObstacleLayer"
        # Whether to enable the obstacle layer
        enabled: True
        # Observation source name
        observation_sources: scan
        scan:
          # Topic name of the laser scan data
          topic: /scan
          # Maximum obstacle height
          max_obstacle_height: 2.0
```

```yaml
        # Whether to clear obstacles
        clearing: True
        # Whether to mark obstacles
        marking: True
        # Data type
        data_type: "LaserScan"
        # Maximum range of ray tracing
        raytrace_max_range: 3.0
        # Minimum range of ray tracing
        raytrace_min_range: 0.0
        # Maximum detection range for obstacles
        obstacle_max_range: 2.5
        # Minimum detection range for obstacles
        obstacle_min_range: 0.0
    voxel_layer:
      # Voxel layer plugin type
      plugin: "nav2_costmap_2d::VoxelLayer"
      # Whether to enable the voxel layer
      enabled: True
      # Whether to publish the voxel map
      publish_voxel_map: True
      # Origin in the Z direction
      origin_z: 0.0
      # Resolution in the Z direction
      z_resolution: 0.05
      # Number of voxels in the Z direction
      z_voxels: 16
      # Maximum obstacle height
      max_obstacle_height: 2.0
      # Marking threshold
      mark_threshold: 0
      # Observation source name
      observation_sources: scan
      scan:
        # Topic name of the laser scan data
        topic: /scan
        # Maximum obstacle height
        max_obstacle_height: 2.0
        # Whether to clear obstacles
        clearing: True
        # Whether to mark obstacles
        marking: True
        # Data type
        data_type: "LaserScan"
        # Maximum range of ray tracing
        raytrace_max_range: 3.0
        # Minimum range of ray tracing
        raytrace_min_range: 0.0
        # Maximum detection range for obstacles
        obstacle_max_range: 2.5
        # Minimum detection range for obstacles
        obstacle_min_range: 0.0
    static_layer:
      # Static layer plugin type
      plugin: "nav2_costmap_2d::StaticLayer"
      # Whether to subscribe to transient local maps
```

```yaml
        map_subscribe_transient_local: True
      inflation_layer:
        # Inflation layer plugin type
        plugin: "nav2_costmap_2d::InflationLayer"
        # Cost scaling factor
        cost_scaling_factor: 3.0
        # Inflation radius
        inflation_radius: 0.3
      # Whether to always send the full costmap
      always_send_full_costmap: True
  global_costmap_client:
    ros__parameters:
      # Whether to use simulation time; False means using actual time
      use_sim_time: False
  global_costmap_rclcpp_node:
    ros__parameters:
      # Whether to use simulation time; False means using actual time
      use_sim_time: False

# Map server configuration
map_server:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False
    # Name of the map YAML file
    yaml_filename: "map.yaml"

# Map saver configuration
map_saver:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False
    # Timeout for saving the map
    save_map_timeout: 5.0
    # Default free threshold
    free_thresh_default: 0.25
    # Default occupied threshold
    occupied_thresh_default: 0.65
    # Whether to subscribe to transient local maps
    map_subscribe_transient_local: True

# Planner server configuration
planner_server:
  ros__parameters:
    # Expected planner frequency
    expected_planner_frequency: 5.0
    # Whether to use simulation time; False means using actual time
    use_sim_time: False
    # List of planner plugin names
    planner_plugins: ["GridBased"]
    GridBased:
      # Grid-based planner plugin type
      plugin: "nav2_navfn_planner/NavfnPlanner"
      # Planning tolerance
      tolerance: 0.5
      # Whether to use the A* algorithm
```

```yaml
      use_astar: false
      # Whether to allow unknown space
      allow_unknown: true

# Planner server RCLCPP node configuration
planner_server_rclcpp_node:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False

# Recovery server configuration
recoveries_server:
  ros__parameters:
    # Topic name of the costmap
    costmap_topic: local_costmap/costmap_raw
    # Topic name of the robot footprint
    footprint_topic: local_costmap/published_footprint
    # Loop frequency
    cycle_frequency: 5.0
    # List of recovery plugin names
    recovery_plugins: ["spin", "backup", "wait"]
    spin:
      # Spin recovery plugin type
      plugin: "nav2_recoveries/Spin"
    backup:
      # BackUp recovery plugin type
      plugin: "nav2_recoveries/BackUp"
    wait:
      # Wait recovery plugin type
      plugin: "nav2_recoveries/Wait"
    # Name of the global coordinate frame
    global_frame: odom
    # Name of the robot's base coordinate frame
    robot_base_frame: base_link
    # Timeout of the transform
    transform_timeout: 0.1
    # Whether to use simulation time; False means using actual time
    use_sim_time: False
    # Look-ahead simulation time
    simulate_ahead_time: 2.0
    # Maximum rotational velocity
    max_rotational_vel: 1.0
    # Minimum rotational velocity
    min_rotational_vel: 0.4
    # Rotational acceleration limit
    rotational_acc_lim: 3.2

# Robot state publisher configuration
robot_state_publisher:
  ros__parameters:
    # Whether to use simulation time; False means using actual time
    use_sim_time: False

# Waypoint follower configuration
waypoint_follower:
  ros__parameters:
```

```
    # Loop rate
    loop_rate: 2000
    # Whether to stop on failure
    stop_on_failure: false
    # Waypoint task executor plugin name
    waypoint_task_executor_plugin: "wait_at_waypoint"
    wait_at_waypoint:
      # Wait-at-waypoint plugin type
      plugin: "nav2_waypoint_follower::WaitAtWaypoint"
      # Whether to enable the plugin
      enabled: True
      # Waypoint pause duration
      waypoint_pause_duration: 200
```

6. Source Code Analysis

- nav2 launch file path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/nav2.launch.py
```

- `nav2_launch_file`: points to the path of nav2's official `bringup_launch.py` launch file
- `nav2_launch_mod`: points to the modified launch file `bringup_mod.launch.py`, which blocks the AMCL localization function and is used for external localization such as Cartographer / slam_toolbox
- `rtabmap_localization_launch_file`: points to the path of the rtabmap localization algorithm launch file
- `carbase`: includes `car_base.launch.py`, enabling the LiDAR and IMU, disabling the camera and magnetometer, and using the EKF odometry as the TF source; it starts only when the localization method is not rtabmap or slam is True
- `amcl_localization_nav`: the AMCL localization navigation group, activated when `localization_mode == 'amcl'` and `slam == 'False'`; it loads the map file and navigation parameter file
- `cartographer_localization_nav`: the Cartographer localization navigation group, activated when `localization_mode == 'cartographer'` and `slam == 'False'`; it starts `cartographer_node` (loads the pbstream state file for localization) and `cartographer_occupancy_grid_node` (generates the grid map)
- `create_rviz_node`: dynamically selects the Rviz configuration file based on `rviz_config` and the camera type, loading `nav2.rviz` by default; it starts only in non-rtabmap mode and when `rviz == 'True'`
- `slam_toolbox_localization_nav`: the slam_toolbox localization navigation group, activated when `localization_mode == 'slam_toolbox'` and `slam == 'False'`; it starts `localization_slam_toolbox_node` for pure localization
- `nav_and_slam`: the mapping-while-navigating group, activated when `localization_mode == 'amcl'` and `slam == 'True'`; it enables AMCL navigation and turns on SLAM mode

- `rtabmap_localization_nav`: the rtabmap localization navigation group, activated when `localization_mode == 'rtabmap'` and `slam == 'False'`; it enables the camera, LiDAR, and IMU at the same time and includes the rtabmap localization launch file

```
    #navigation2 startup file
    nav2_launch_file = os.path.join(get_package_share_directory("nav2_bringup"),
                                    "launch",
                                    'bringup_launch.py')

    #The modified startup file of navigation2 blocks the AMCL positioning
function
    nav2_launch_mod = os.path.join(bringup_dir,
                                    'launch',
                                    'bringup_mod.launch.py'
                                    )

    #rtabmap positioning algorithm startup file
    rtabmap_localization_launch_file=os.path.join(bringup_dir,
                                        'launch',
                                        'rtab_map.launch.py')

    carbase=IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(bringup_dir,'launch','car_base.launch
.py')),
            condition=IfCondition(PythonExpression(["(", "'", localization_mode,
"' != 'rtabmap' or '", slam, "' == 'True' ) and '", start_carbase, "' ==
'True'"])),
            launch_arguments={'log_level': 'info',
                              'use_imu': 'True',
                              'use_lidar': 'True',
                              'use_mag': 'False',
                              'use_camera': 'False',
                              'odom_tf_source': 'ekf'
                              }.items())

    #AMCL positioning method navigation
    amcl_localization_nav = GroupAction(
        condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'amcl' and '", slam, "' == 'False' "])),
        actions=[
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(nav2_launch_file),
                launch_arguments={
                    "map": map_file,
                    "params_file": params_file,
                    "use_sim_time": "False",
                    "log_level": log_level,
                }.items(),
            )
        ]
    )

    #Cartographer positioning navigation
    cartographer_localization_nav = GroupAction(
```

```
        condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'cartographer' and '", slam, "' == 'False' "])),
        actions=[
            #Start navigation2 navigation file (no global positioning)
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(nav2_launch_mod),
                launch_arguments={
                                "map": map_file,
                                "params_file": params_file,
                                "use_sim_time": "False",
                                "log_level": log_level,
                                }.items()),
            Node(
                package =   'cartographer_ros',
                executable = 'cartographer_node',
                parameters = [{'use_sim_time': False}],
                arguments = [
                            '-configuration_directory',
os.path.join(get_package_share_directory('yahboomcar_bringup'),'param'),
                            '-configuration_basename',
'cartographer_2d_localization.lua',
                            '-load_state_filename', state_filename_file],
                remappings = [('imu', '/imu/data_raw')],
                output = 'screen'
                ),
            Node(
                package = 'cartographer_ros',
                executable = 'cartographer_occupancy_grid_node',
                parameters = [
                    {'use_sim_time': False},
                    {'resolution': 0.05}],
                )
        ]
    )

    #rviz
    def create_rviz_node(context, *args, **kwargs):
        rviz_config_value = LaunchConfiguration('rviz_config').perform(context)
        camera_type = GetCameraType()
        if rviz_config_value == 'roadnet_edit':
            if camera_type == 'usb':
                file_name = 'roadnet_edit_usb.rviz'
            elif camera_type == 'nuwa':
                file_name = 'roadnet_edit.rviz'
            rviz_config_file = os.path.join(bringup_dir, 'rviz', file_name)
        elif rviz_config_value == 'record_tag':
            if camera_type == 'usb':
                file_name = 'aritag_record_usb.rviz'
            elif camera_type == 'nuwa':
                file_name = 'aritag_record.rviz'
            rviz_config_file = os.path.join(bringup_dir, 'rviz', file_name)
        else:
            rviz_config_file = os.path.join(bringup_dir, 'rviz', 'nav2.rviz')

        rviz_node = Node(
            package='rviz2',
```

```
            executable='rviz2',
            name='rviz2',
            arguments=['-d', rviz_config_file,'--ros-args','--log-level','warn'],
            output='screen',
            condition=IfCondition(PythonExpression(["'", localization_mode, "' !=
'rtabmap' and '", rviz, "' == 'True' "]))
        )
        return [rviz_node]


    slam_toolbox_localization_nav = GroupAction(
        condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'slam_toolbox' and '", slam, "' == 'False' "])),
        actions=[
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(nav2_launch_mod),
                launch_arguments={
                                "map": map_file,
                                "params_file": params_file,
                                "use_sim_time": "False",
                                "log_level": log_level,
                                }.items()),
            Node(
                package='slam_toolbox',
                executable='localization_slam_toolbox_node',
                name='slam_toolbox',
                parameters=[

 os.path.join(bringup_dir,'param','mapper_params_localization.yaml'),
                  {'use_sim_time': False},
                  {'map_file_name': pose_map}
                ],
                arguments=['--ros-args','--log-level','warn'],
                output='screen')
        ]
    )
    #Navigate while building the map
    nav_and_slam = GroupAction(
        condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'amcl' and '", slam, "' == 'True' "])),
        actions=[
            # Navigation using AMCL positioning method
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(nav2_launch_file),
                launch_arguments={
                    "map": map_file,
                    "use_sim_time": "False",
                    "params_file": params_file,
                    'slam': 'True',
                }.items(),
            )
        ]
    )

    #rtabmap positioning navigation
    rtabmap_localization_nav = GroupAction(
```

```
        condition=IfCondition(PythonExpression(["'", localization_mode, "' ==
'rtabmap' and '", slam, "' == 'False' "])),
        actions=[
            # Start car_base
            IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(bringup_dir,'launch','car_base.launch
.py')),
                launch_arguments={'log_level': 'info',
                                  'use_imu': 'True',
                                  'use_lidar': 'True',
                                  'use_mag': 'False',
                                  'use_camera': 'True',
                                  'odom_tf_source': 'ekf'
                                  }.items()),
            # Start navigation2 navigation file (no global positioning)
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(nav2_launch_mod),
                launch_arguments={
                                "map": map_file,
                                "params_file": params_file,
                                "use_sim_time": "False",
                                }.items()),
            IncludeLaunchDescription(
                PythonLaunchDescriptionSource(rtabmap_localization_launch_file),
                launch_arguments={
                    "mode": "localization",
                }.items(),
            )
        ]
    )

    return LaunchDescription(
        [
            *declared_arguments,
            OpaqueFunction(function=create_rviz_node),
            carbase,
            amcl_localization_nav,
            cartographer_localization_nav,
            slam_toolbox_localization_nav,
            rtabmap_localization_nav,
            nav_and_slam
        ]
    )
```